

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

1.3

HOW IT WORKS

 Copy as Markdown

How Serverpod works

With Serverpod, you write type-safe Dart on both your Flutter app and the backend. Edit a file and hit save. Serverpod regenerates the code that connects your server and Flutter app. It then hot reloads both while they run. To change the database schema, you press one key to create and apply a migration. No manual rebuilds, no Docker to set up, and no API code to write by hand.

Serverpod is a full backend. It manages your database, authentication, file uploads, caching, real-time communication, scheduling, and logging. You can focus on building features instead of wiring together separate services. A project is a single workspace of Dart packages, and a code generator keeps the types shared between your server and app in sync. If you rename a field or use an incorrect type, they will show up as compile-time errors rather than a surprise when you run the app.

Create your project

A Serverpod project starts with the `serverpod create` command, which walks you through a few choices that shape what it generates:

- **Project type:** A server with a Flutter app (the default), a server only, or a reusable [module](#) shared across servers.
- **Database and caching:** Add a Postgres database and Redis (for pub/sub and caching).
- **Authentication:** Built-in email and social sign-ins.
- **Web server:** Optionally serve web pages and your Flutter web app alongside your API.
- **Code editors and AI agents (optional):** Pick any of Antigravity, Claude, Codex, Cursor, OpenCode, and VS Code. Serverpod installs skills and MCP servers for each one you select.

The result is one workspace. For the default project type, it has three Dart packages:

```
my_project/  
├─ my_project_server/  # Your backend code.
```



```
|— my_project_client/    # Generated client code used by the app.
|— my_project_flutter/  # Your Flutter app.
```

Server-only projects and modules have the same layout without the `_flutter` package.

In the `_server` package you add your endpoints and data models. Serverpod's code generator generates code on the server and in the client package. You get a type-safe Dart API for your app, along with the serialization and database code on the server. You never write serialization, HTTP calls, or API contracts. For what each package holds in detail, see [Your Serverpod project](#).

Run your project

You run the whole project with one command:

```
$ serverpod start
```

This starts your server, launches your Flutter app, and watches your code. When you make a change, Serverpod instantly regenerates the client, hot reloads the server, and reloads your app.

TIP

Start your project before you begin building. With `serverpod start` already running, every change after that, whether you make it yourself or an agent does it, is reloaded as you go.

Write an endpoint

In Serverpod, endpoints are the entry points your app calls to run code on the server. You define one as a class that extends `Endpoint`, with async methods that each take a `Session` as their first argument and return a typed `Future` (or a `Stream`, for live updates):

```
class ExampleEndpoint extends Endpoint {
  Future<String> hello(Session session, String name) async {
    return 'Hello $name';
  }
}
```

The `Session` is the context for that one call, with access to the database, cache, signed-in user, and logging. Parameters and return types can be the built-in types (`bool`, `int`, `double`, `String`, `DateTime`, `Duration`, `Uri`, `UuidValue`, `BigInt`, `ByteData`), a `List`, `Map`, `Set`, or `Record` of those, or any data model you define. See [Working with endpoints](#) for more information.

Call it from your app

On the app side, the generated client turns each endpoint method into what looks like a local method:

```
final greeting = await client.example.hello('World');
```

The client handles the request, the response, and the JSON in between. Most calls follow this request-and-response shape. For live updates, Serverpod also has [streaming methods](#) that keep a connection open so the server and app can push data to each other.

Define your data models

Models are the data classes your app and server pass back and forth, the objects your endpoints take and return. You define them in `.spy.yaml` files, each naming a class and listing its fields:

```
class: Company
fields:
  name: String
  foundedDate: DateTime?
```

When you generate code, this becomes a `Company` Dart class shared by the server and your app, so the same type flows from your endpoints into your Flutter widgets. Fields support the same types as endpoint methods, plus enums and nested models, and can be nullable. See [Working with models](#) for the details.

Add a `table` key to also store the model in a database table:

```
class: Company
table: company
fields:
  name: String
  foundedDate: DateTime?
```

The generated `Company` class then gains a `db` field with type-safe methods for reading and writing rows, so you query the database in Dart instead of SQL. For example, insert a row and read it back:

```
final company = await Company.db.insertRow(session, Company(name: 'Serverpod'));
final stored = await Company.db.findById(session, company.id!);
```

These run on the same `session` your endpoint method receives. When you change a table model, press `M` in the `serverpod start` terminal to create the migration and apply it. Pending migrations also apply on startup.

That database runs without setup on your part: Serverpod manages an embedded Postgres for you, with no Docker to configure. If you would rather manage Postgres yourself, you can change the configuration in the server's `config` directory.

See [CRUD](#) for the generated query methods, [relations](#) for linking tables, and [transactions](#) for atomic operations.

Build with an AI agent

Serverpod is built to work with AI coding agents. You pick your agent when you create the project, and Serverpod installs skills that teach it how a Serverpod project fits together. While `serverpod start` runs, it exposes an MCP server that lets the agent work with your live project. It can write endpoints and models, manage the database, and reload the app as it goes. You describe the feature you want, and the agent builds it.

Your data models and endpoint methods are the single source of truth. When the agent renames a field or changes its type, the regenerated code treats the mismatch as a compile-time error rather than a runtime bug.

If you want to quickly try out Serverpod and build something with an AI agent, check out our [Quickstart](#) guide.

Next steps

Ready to build something for real? Follow [Build your first app](#) to create your first endpoint and call it from Flutter. When you are ready to ship, [Deploy to Serverpod Cloud](#) takes your app to production in just a few minutes.